Smart MCQ Solver Challenge

Project Report

Detail	Value
Roll Number	23F2003665
GitHub Repository	dl-genai-project-26-t2
W&B Project	23f2003665-t22026

1. Abstract / Executive Summary

The Smart MCQ Solver Challenge requires ranking five answer options (A-E) and submitting the top three for each question using MAP@3.

I compared three approaches: TF-IDF + Logistic Regression, a TextCNN trained from scratch in PyTorch, and a pretrained DistilBERT model fine-tuned with Hugging Face. Their Kaggle MAP@3 scores were 0.75103, 0.74522 and 0.75561 respectively, with DistilBERT giving the best score among the documented models. EDA also revealed 242 duplicate prompt rows and 54.3% overlap between unique train and test prompts, showing why random-split validation can be overly optimistic.

2. Introduction

The task is to rank options A-E for each MCQ and return the best three in order. The report covers data analysis, tokenization, model design, training, evaluation and final conclusions.

3. Dataset & Preprocessing

Item	Finding
Training set	2,000 questions, 8 columns
Test set	500 questions, 7 columns
Missing values	None in train or test
Duplicate prompt rows	242
Unique prompts	Train 1,758 | Test 492
Train-test prompt overlap	267 unique prompts (54.3% of test prompts)
Prompt length	Mean 117.67, median 111, max 337 characters

3.1 Exploratory Data Analysis

The answer distribution is mildly imbalanced: B 24.50%, C 22.95%, A 18.45%, D 17.90% and E 16.20%. Prompt length was measured in characters. The most important finding was repeated content: duplicate prompts can appear across a normal random validation split, and many test prompts also occur in training after simple normalization.

3.2 Preprocessing & Data Augmentation

Preprocessing was model-specific. TF-IDF and TextCNN used lowercase text with punctuation/special-character cleanup and whitespace normalization. DistilBERT used prompt-option pairs and removed repeated instruction phrases such as 'choose the correct answer' while normalizing text and whitespace. No synthetic data augmentation was used. Splits were made before pair expansion so the five options of one MCQ remained together.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best model, DistilBERT, used the pretrained tokenizer associated with distilbert-base-uncased. It converts prompt-option sentence pairs into subword input IDs and an attention mask. This tokenizer matches the pretrained model vocabulary and therefore preserves compatibility with the learned language representations.

4.2 Implementation & Comparison

DistilBERT pairs were tokenized in batches, padded/truncated to 160 positions and converted to PyTorch tensors. TextCNN instead used a custom word-level vocabulary with <PAD> and <UNK> tokens and learned 64-dimensional embeddings from scratch. TF-IDF used no neural token embeddings; it represented cleaned prompt-option text using sparse n-gram features.

5. Modeling & Experimentation

5.1 TF-IDF + Logistic Regression

Each original question was split 80/20 before expansion into five prompt-option pairs. Correct pairs were labeled 1 and incorrect pairs 0. The cleaned prompt and option were joined, transformed by TF-IDF, and scored with Logistic Regression. For each question, the five P(class=1) values were sorted to select the top three.

Parameter	Value
Split	80/20, random_state=42
TF-IDF	ngram_range=(1,3), sublinear_tf=True
Maximum features	50,000
Logistic Regression	max_iter=5,000, class_weight=balanced
Validation MAP@3	0.9938
Kaggle MAP@3	0.75103

The local validation score is much higher than Kaggle performance because duplicate prompt content can cross a normal random split. The model is nevertheless competitive because TF-IDF captures useful lexical patterns and Logistic Regression converts them into pairwise correctness probabilities.

5.2 TextCNN - Custom Deep Learning Model

TextCNN uses one combined sequence containing the prompt and all five options. Words are mapped to a custom vocabulary, padded/truncated to 160 tokens and passed through a small CNN that directly produces five answer logits.

5.2.1 Architecture

Layer / Step	Configuration	Purpose
Embedding	vocab_size x 64, padding_idx=0	Learn word representations
Conv1D	64 input channels, 64 filters, kernel=3	Detect local three-token patterns
ReLU	max(0,x)	Add non-linearity
Global Max Pool	Max over sequence	Keep strongest activation per filter
Linear	64 -> 5	Produce logits for A-E

5.2.2 Training Strategy

The answer letters were mapped to classes A->0, B->1, C->2, D->3 and E->4. A stratified 85/15 split preserved class proportions. Mini-batches of 32 were trained for 10 epochs with CrossEntropyLoss and Adam. In every batch, the model performs a forward pass, computes loss, backpropagates gradients with loss.backward(), and updates weights with optimizer.step().

Parameter	Value
Maximum length	160 word tokens
Batch size	32
Embedding dimension	64
Filters / kernel	64 / 3
Loss	CrossEntropyLoss
Optimizer / LR	Adam / 1e-3
Epochs	10
Kaggle MAP@3	0.74522

5.2.3 Validation & Inference

After each epoch the model was evaluated without gradient calculation. Softmax made the five logits easier to interpret, and the top three indices were used for MAP@3. The best state_dict was saved when validation MAP@3 improved. For test inference, the same preprocessing was applied and the best checkpoint produced the final A-E ranking.

5.2.4 Salient Points

- The embedding layer and convolution filters are learned entirely from the project data; no pretrained weights are used.
- Kernel size 3 focuses on short local patterns, while global max pooling keeps the strongest feature detected anywhere in the sequence.
- The architecture is compact and interpretable, but it has less contextual capacity than a pretrained Transformer.

5.3 Fine-Tuned Transformer - DistilBERT

5.3.1 Architecture & Pairwise Formulation

DistilBERT is an encoder-only Transformer obtained through knowledge distillation from BERT. The base model has 6 Transformer encoder blocks, hidden size 768 and 12 self-attention heads. For this project, each MCQ becomes five (prompt, option) pairs. The sequence-classification head uses the final first-token representation to output one scalar relevance score per pair.

5.3.2 Tokenization & Fine-Tuning

Repeated instruction phrases were removed from prompts, then original rows were split 90/10. Correct pairs received target 1.0 and incorrect pairs 0.0. The pretrained tokenizer created input_ids and attention_mask. With num_labels=1 and problem_type='regression', the scalar output was trained using Mean Squared Error. Hugging Face Trainer handled optimization, evaluation and checkpoint saving; load_best_model_at_end=True restored the checkpoint with the highest validation MAP@3.

Parameter	Value
Base model	distilbert-base-uncased
Split	90/10, random_state=42
Max length / batch	160 / 16
Epochs / learning rate	10 / 2e-5
num_labels / problem type	1 / regression
Loss	MSE
Evaluation / save	Every epoch
Kaggle MAP@3	0.75561

5.3.3 Validation & Test Inference

Pair logits were reshaped to (-1, 5), relying on A-E pair order so each row corresponded to one original question. The five scores were sorted and the top three compared with the known validation answer. Test inference used the saved best model and the same reshaping/ranking procedure to produce the submission.

5.3.4 Why DistilBERT?

Self-attention allows each token to build a contextual representation using information from the entire prompt-option sequence. Compared with the from-scratch CNN, DistilBERT starts with pretrained language representations and therefore models richer semantic relationships while remaining smaller than BERT-base.

6. Performance & Comparative Analysis

6.1 Evaluation Metric - MAP@3

MAP@3 rewards both correctness and rank. For a single-answer MCQ, a correct answer at rank 1 scores 1.0, rank 2 scores 0.5, rank 3 scores 1/3, and an answer outside the top three scores 0. The final score is the mean across all questions.

Correct answer position	Per-question score
Rank 1	1.000
Rank 2	0.500
Rank 3	0.333
Not in top 3	0.000

6.2 Comparative Report

Type	Method	Kaggle MAP@3
Baseline(Top 3)	Most-frequent B C A	0.4213
Baseline(Random)	Random top-3	0.3675
Classical ML	TF-IDF + Logistic Regression	0.75103
Custom DL	TextCNN	0.74522
Pretrained Transformer	DistilBERT	0.75561

6.3 Training Performance & W&B

Weights & Biases was used to record training information for the neural models. The tracked quantities included training loss, evaluation loss and validation MAP@3, together with run configuration such as model name, learning rate, batch size, epochs and maximum input length. These logs were used to inspect training progress and compare runs.

6.4 Kaggle Performance & Main Findings

- DistilBERT achieved Kaggle MAP@3: 0.75561.
- TF-IDF + Logistic Regression remained highly competitive at 0.75103 despite being much simpler and cheaper to train.
- TextCNN reached 0.74522 with a compact one-convolution architecture trained from scratch.
- All three trained approaches clearly outperformed the non-learning baselines.
- Very high local validation scores should be interpreted cautiously because repeated prompt content can cross a random split.

6.5 Evaluation Limitation

The largest evaluation risk is duplicate-content leakage. The dataset contains 242 duplicate prompt rows and 267 unique prompts shared between train and test. A random validation split can therefore contain highly similar or repeated questions on both sides, inflating local MAP@3. The Kaggle leaderboard was treated as the more realistic external comparison.

7. Conclusion & Future Work

The project compared three distinct approaches to the same ranking task. TF-IDF + Logistic Regression used sparse lexical features and pairwise probabilities; TextCNN learned embeddings and local phrase patterns from scratch; DistilBERT used pretrained contextual representations and pairwise relevance scoring. DistilBERT gave the best documented Kaggle result, while the classical model remained surprisingly close. The main methodological lesson was that data quality and validation design matter as much as architecture choice.

7.1 Key Learnings

- How TF-IDF features and Logistic Regression turn text pairs into correctness probabilities.
- How embeddings, Conv1D, ReLU, global max pooling, CrossEntropyLoss and backpropagation work together in TextCNN.
- How pretrained tokenization, self-attention, Transformer encoders and fine-tuning are used in DistilBERT.
- How MAP@3 evaluates ranked top-three predictions rather than a single hard class.
- How duplicate content and train-test overlap can create misleading validation scores

.

7.2 Challenges Faced

- Duplicate prompts and train-test overlap made local validation scores difficult to trust.
- Different model families required different input representations: sparse n-grams, custom word IDs and pretrained subword tokens.
- GPU/compute limits required compact model choices and careful checkpoint-based inference.

7.3 Areas for Improvement

- Use grouped or duplicate-aware validation so the same normalized prompt cannot appear in both training and validation sets.
- Compare additional pretrained encoders and tune learning rate, sequence length and regularization systematically.
- Experiment with ensembling the strongest models to combine lexical and contextual signals.

8. References

Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP 2014.
Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT. NeurIPS Workshop.
Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.
Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.
Project software: PyTorch, scikit-learn, Hugging Face Transformers/Datasets, and Weights & Biases.